

HomeWork 3 Laboratorio Bioimmagini

Luca Perrone

10 Gennaio 2024

1 Exercise 1 and 2

1.1 Exercise 1

After loading the images dataset downloaded from Zenodo portal, data was filtered in order to keep only images labeled as: s “fetal femur”, “fetal brain” and “maternal cervix”.

It was noticed that the Fetal Brain class (composed of 3092 elements) had almost double the element of the Maternal cervix one, and three times the Fetal femur ones. So, keeping this in mind, a data augmentation was performed to make Maternal cervix data double in size and three times for Fetal Femour. in the end, to make the set more omogeneous, the three classes had been cut to have the same number of values as the less populated class.

So the train and test split was performed in order to use 70% of the set for training (2164 images for each class) and the rest for test (928 images per class).

To get the feature size for Histogram of Oriented Gradients for both [4x4] and [8x8] cell-sizes and build the features matrix, both HOGs were computed on the first image and the length was saved in the variables *hogFeatureSize4* and *hogFeatureSize8*.

A **for** loop was used to iterate on the training image set and extract the HOG features to feed the SVMs classifiers: *classifier4* was fed with [4x4] HOG features and *classifier8* using the [8x8] features.

The same extraction procedure was executed to retrieve features from the test set but this time were fed to the respective trained classifier and to predict the belonging class.

Both for traing and testing, images has been imported using *readAndPreprocessImage(filename)* custom function to resize them to 227x227 pixels.

To evaluate the classification results, for both classifier a **Confusion Matrix** was built and the accuracy for each class was evaluated.

Accuracy values obtained with [4x4] HOG cell size:

$$ConfusionMatrix[4x4] = \begin{bmatrix} 0.8524 & 0.1272 & 0.0205 \\ 0.0506 & 0.9203 & 0.0291 \\ 0.0636 & 0.0172 & 0.9192 \end{bmatrix}$$

- Accuracy for class 1: 0.88183
- Accuracy for class 2: 0.86437
- Accuracy for class 3: 0.94883
- Mean accuracy: 0.89727

Accuracy values obtained with [8x8] HOG cell size:

$$ConfusionMatrix[8x8] = \begin{bmatrix} 0.8200 & 0.1261 & 0.0539 \\ 0.0453 & 0.9289 & 0.0259 \\ 0.0593 & 0.0334 & 0.9073 \end{bmatrix}$$

- Accuracy for class 1: 0.88695
- Accuracy for class 2: 0.85347
- Accuracy for class 3: 0.91921
- Mean accuracy: 0.88542

1.2 Exercise 2

The code for this exercise was mostly the same of the previous one but this time the features have been extracted using **AlexNet**. Also, in addition to the SVM classifier, Decision Tree classifier was trained with the same extracted features .

Accuracy values obtained with SVM classifier:

$$ConfusionMatrixSVM = \begin{bmatrix} 0.9558 & 0.0345 & 0.0097 \\ 0.4688 & 0.3869 & 0.1444 \\ 0.1897 & 0.0032 & 0.8071 \end{bmatrix}$$

- Accuracy for class 1: 0.59212
- Accuracy for class 2: 0.91117
- Accuracy for class 3: 0.83969
- Mean accuracy: 0.71659

Accuracy values obtained with Decision Tree classifier:

$$ConfusionMatrixDT = \begin{bmatrix} 0.7231 & 0.1681 & 0.1088 \\ 0.0614 & 0.8944 & 0.0442 \\ 0.0582 & 0.0787 & 0.8631 \end{bmatrix}$$

- Accuracy for class 1: 0.85806
- Accuracy for class 2: 0.78376
- Accuracy for class 3: 0.84942
- Mean accuracy: 0.82687

1.3 Results

The classification results demonstrated a notable improvement in accuracy across all classifiers, with an approximate increase of 0.1 in mean accuracy for each classifier following the implementation of data augmentation. However, it is important to note that this enhancement in accuracy comes at the expense of longer computation times.

Additionally, the classifiers exhibited slight variations in accuracy values when classifying different classes. This observed variability may be attributed to the data augmentation process. Classifying non-augmented data tended to yield lower accuracy values but potentially provided the models with greater generalization capabilities.

2 Exercise 3

The process for this exercise begun from the segmentation of the thorax through the use of a threshold to extract the binary mask. An opening function was used to smooth the edges and a area opening function to remove small objects in the image, assuming that the thorax should occupy most of the space in the image. An inverse of this mask was also saved to be later used in the lungs segmentation to remove background.

For the segmentation of the lungs an opening function has been performed on the slices through which the binary mask was extrapolated. On this binary image a strong closing was used to close small holes and imperfections from the mask and a milder opening to clean the area inside the lungs. Then the background was removed using the previously computed thorax mask.

To make sure that the detected objects in the image were actually lungs, the mask was also flipped and summed to the mirrored mask to check if any overlapping pixels were present: it was assumed that if an object in the image was a lung, there should have been another object in the image that had at least one symmetric pixel (the other lung).

So the perimeter of the lungs in each slice was calculated using *bwperim()* function.

To calculate the volume in pixel of the two lungs separately two translation were performed: one to the left, one to the right, (both in x-direction and with a traslation vector equal to half the width of the image) to isolate the two lungs and sum white pixels in the resulting images.

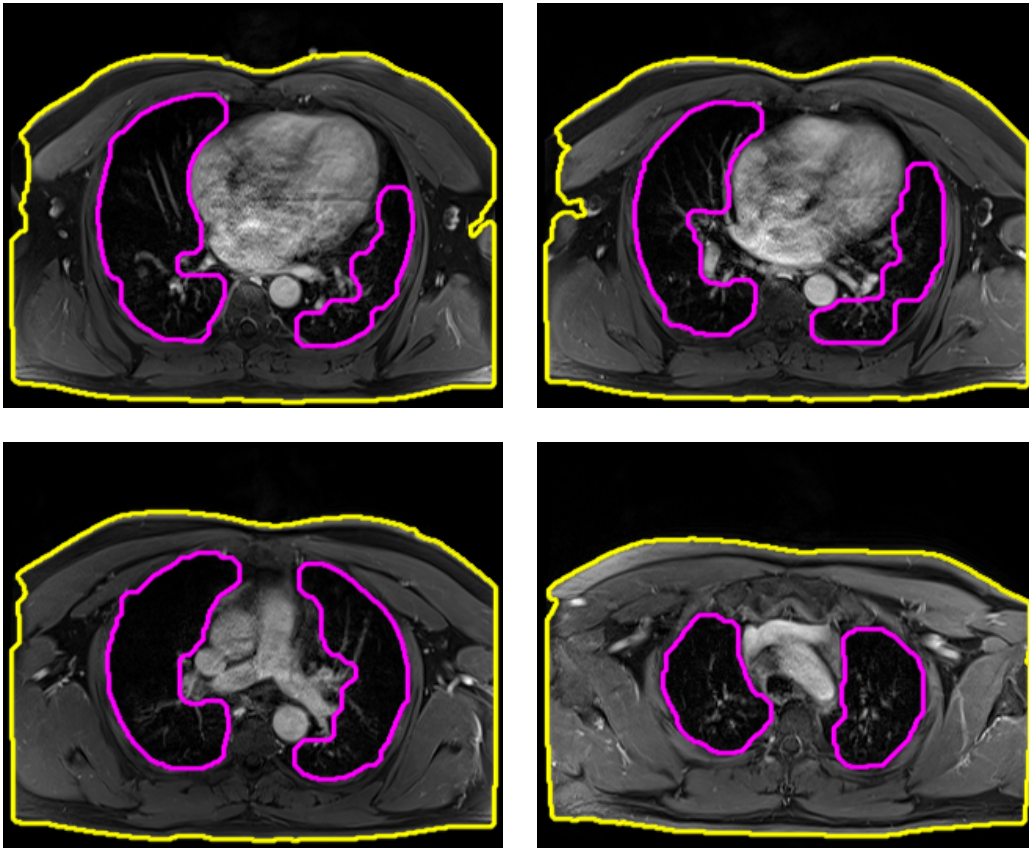


Figure 1: Examples of segmentation results